Interrupts and Exceptions

Reminders

- Interrupts

- Quiz due Sunday
  - **On this weeks materials**

- Demo due Friday
  - **On interrupts!**
  - **Step through provided program**
    - Try stepping into function call

Basics

- What is an interrupt?
  - **A hardware or software event that causes the microprocessor to immediately halt execution of the currently running program and transfer control to code that responds to the event**
- Why do we need interrupts?
  - **Input/output devices are slower than memory and CPU**
  - **Devices require fast response from CPU when events occur, even when the CPU is busy running a program**
  - **Uncertainty of when device will be ready**
    - Saves time from polling device (polling is the process of continuously checking a signal to see if it has changed)

Basics

- Types of Interrupts
  - **External Interrupts**
    - Used by CPU to interact with input/output devices
      - ***Ex: CPU requests a data transfer and external device asserts an interrupt when it is ready***
    - Improves system performance by allowing the CPU to execute instructions without having to use resources to continuously check to see if I/O device is ready.
  - **Internal Interrupts**
    - Occurs entirely within the CPU
    - Used to handle exceptions that occur during the execution of instructions
    - Ex: Divide by zero exception, Arithmetic overflow, Invalid instruction codes

Basics

- Types of Interrupts (con't)
  - **Software Interrupts**
    - Generated by specific interrupt instructions in the CPU's instruction set
    - Act like subroutine call statements except go to a specific address

Interrupts and Exceptions

- Types of Interrupts Triggers
  - **Level Trigger**
    - Interrupt signaled by steady state of line (either ‘1’ or ‘0’)
    - As long as signal remain asserted an interrupt would be issued
  - **Edge Trigger**
    - Interrupts signaled by a level transition on line
      - ***Either falling edge of rising edge***
- Classification of Interrupts
  - **Maskable Interrupts**
    - Masking means that the interrupt can be ignored
    - Masking is controlled by software
  - **Non-Maskable Interrupt (NMI)**
    - Means that the interrupt <u>cannot</u> be ignored
    - System critical events are attached to NMI

NIOS II Specific Terminology

NIOS II

- Terminology
  - **Exception**
    - A transfer of control away from a program's normal flow of execution, caused by an event, either internal or external to the processor, which requires immediate attention. Exceptions include software & hardware exceptions.
  - **Interrupts**
    - An exception caused by an explicit hardware request signal from an external device. A hardware interrupt diverts the processor's execution flow to an ISR, to ensure that a hardware condition is handled in a timely manner.
  - **No clear consensus as to the exact meaning of these**
    - Different manufacturers use terms like *exceptions, faults, aborts, traps, and interrupts*
    - We use them interchangeably

General Terminology

- Terminology
  - **Imprecise Exception**
    - Exception in pipeline processors not associated with an exact instruction that caused exception
      - ***Ex: bus error cannot be associated with a specific instruction***
  - **Precise Exceptions**
    - An exception in pipeline processors that can be associated with an exact instruction
    - Processor completed execution all instructions preceding faulting instruction
      - ***Return address either the instruction that caused exception or next instruction***
      - ***Execution of instructions following the faulting have not started***
      - ***Allows processor to resume execution after exception cleared***

General Terminology

- Terminology
  - **Synchronous Exception**
    - Caused by instruction execution or an attempt to execute
  - **Asynchronous Exception**
    - Not related to instruction being executed
    - Occur independently of instruction execution
      - ***Ex: From external source, such as I/O device***
  - **Interrupt Priority**
    - Mechanism to determine which interrupt gets CPU if multiple interrupts are asserted at same time
    - Each interrupt assigned a number that represents priority
    - Want critical task to have higher priority

General Terminology

- Terminology
  - **Interrupt Latency**
    - Amount of time system takes to respond to an interrupt
      - ***Time between assertion of interrupt request and executing 1st instruction at exception address***
      - ***Typically defined in CPU clock cycles***
    - Latency depends on
      - ***Priority level of interrupt request***
      - ***Current interrupt request pending***
      - ***Length of current interrupt request being serviced***

General Terminology

- Terminology
  - **Interrupt Response Time**
    - Time from when interrupt is first generated to when processor runs 1st instruction in ISR (Interrupt Service Routine – defined on the next slide)
    - Embedded Systems want short/ deterministic response
      - ***Required for real-time response***
      - ***Minimum interrupt latency tolerance***
        - » How long can a request be ignored without problem?
          - Ex: FIFO filling before FIFO overflows and data lost
  - **Interrupt Recovery Time**
    - Time from when interrupt is first generated to when processor runs last ISR instruction and resumes normal processing

Interrupt Service Routine

- Interrupt Service Routine (ISR)
  - **Code that is executed upon reception of interrupt**
  - **Interrupt forces next instruction to be a call to predetermined location**
  - **Interrupts take time away from processor**
  - **ISR code that processes the interrupt event should**
    - Save and restore any registers used within the ISR
    - Perform minimum work possible in the interrupt handler
      - ***Interrupt service routine (ISR) performs critical work***
      - ***Read/Write critical register values (control and IO)***
      - ***Set flag to another process/thread to complete non-critical work***
      - ***Goal is to get out quickly so the main program can continue execution***
    - Acknowledge interrupt request to clear interrupt event
      - ***Should be last thing done before leaving ISR***

Interrupt Service Routine

- What happens when a program transfers control to ISR?
  - **Parameters must be placed where ISR can access them**
  - **Save return address of next main program instruction to be executed**
  - **ISR acquires storage resources for variables**
  - **ISR performs task**
  - **Results are placed where calling program can access them**
  - **Return control to original point in program**

- ISR call must not impact calling program!
  - **Context or state of processor must be saved**
    - Requires saving registers that might be changed to stack (temporary storage)

NIOS II Specifics

- Nios II Exception Types
  - **Reset exceptions**
    - Occurs when the Nios II processor is reset
    - Control transfers to the reset address defined by user
  - **Break exceptions**
    - Occurs when the JTAG debug module requests control
    - Control transfers to the break address defined by user
  - **Interrupt exceptions**
    - Occurs when peripheral device signals event requiring service
  - **Instruction-related exceptions**
    - Occurs when any instruction related conditions occurs
      - ***Ex: TLB miss, Illegal instruction, MPU violation, etc***
    - Control transfers to the exception address defined by user

NOTE: All Nios II exceptions are precise

Reset

- Reset Exception
  - **Clears all fields of the status register and sets RSIE to 1**
    - Register-set interrupt enable
    - Disables hardware interrupts
    - Puts the processor in Supervisor mode
  - **Invalidates instruction cache**
    - Guarantees code fetched from memory
  - **Starts executing reset handler at reset vector**
    - All general purpose registers are undefined
    - All control registers reset to defaults

NIOS II Specifics

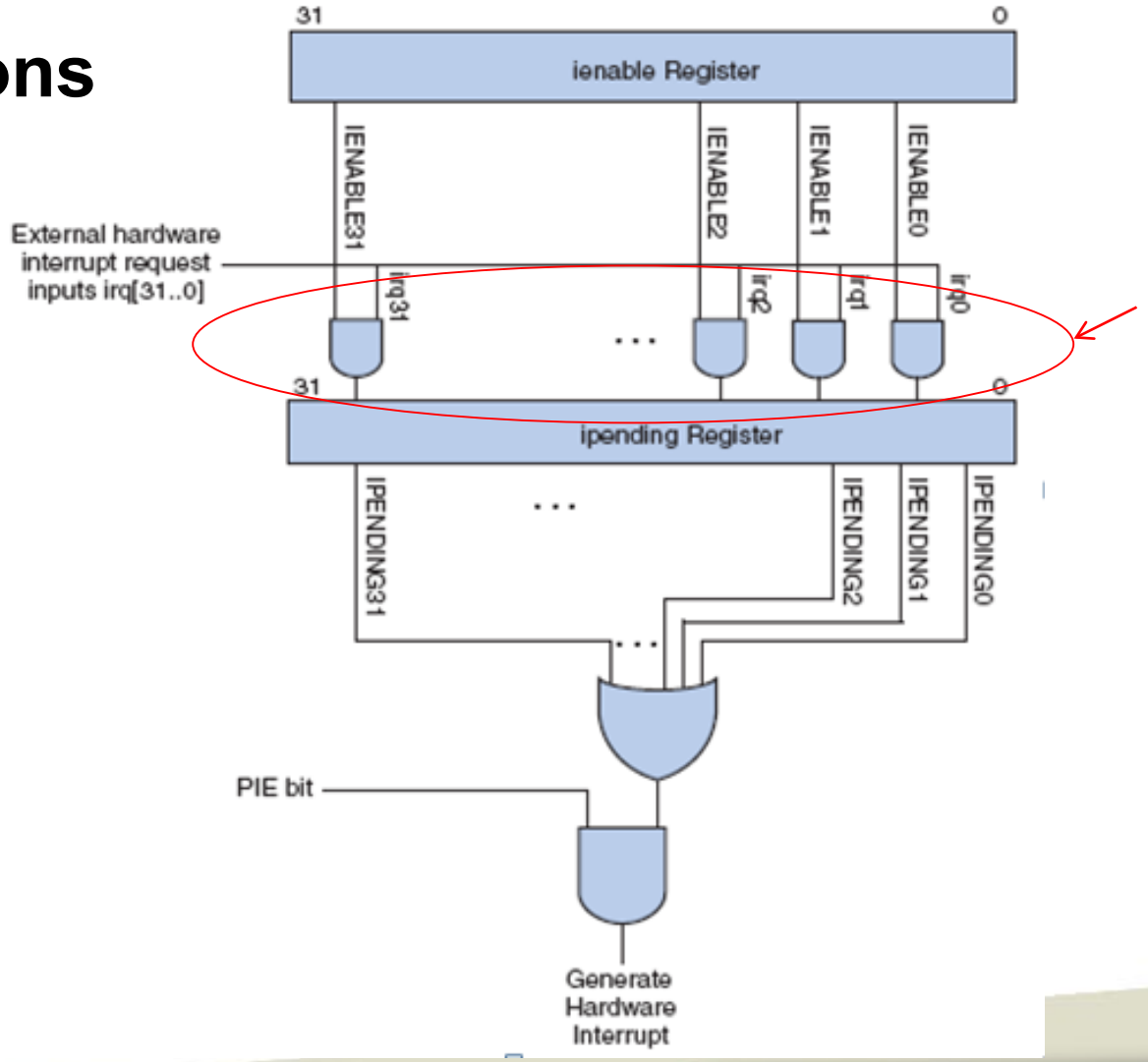
- **Interrupt Exceptions**
  - **Can use internal or external interrupt controller**
    - Internal
      - ***Nios II has 32 level-sensitive interrupt request irq[31-0]***
      - ***Upon interrupt, the processor clears the processor interrupt enable (PIE) bit to zero***
      - ***Supports nested interrupts.***
    - External
      - ***Only on the Nios II/f core***
      - ***Uses Requested Interrupt Level (RIL) to decide when to take a maskable interrupt***
      - ***Interrupt is taken only when the RIL is greater than status.IL.***

NIOS II Specifics

- **Interrupt Exceptions**
  - **Assertion of external interrupt-request inputs**
  - **Register reserved for interrupt exception**
    - PIE bit in ***status*** register
      - ***Global interrupt enable***
    - ***ienable*** Register
      - ***Used to enable interrupt to Nios II***
      - ***Think of this as the interrupt mask register***
    - ***ipending*** Register
      - ***Used to determine interrupt asserted to Nios II***
  - **Interrupt generated only if**
    - **PIE** bit in ***status*** control register set
    - Interrupt-request input (irq*n*) *asserted*
    - irq*n* bit *n of* ***ienable*** *control register set*

NIOS II Specifics

- **Interrupt Exceptions**



Gates off external signal from reaching ipending register

NIOS II Interrupt coding

R·I·T

NIOS II

- Hardware Interrupts
  - **Before ISR function can be used you must**
    - Register ISR function with interrupt
      - ***Links interrupt to ISR function***
      - ***Easiest if HAL function `alt_ic_isr_register()` is used***
        - » See example in demo
    - *Clear interrupt status bit using **ipending** register*
      - ***Ensures interrupt clear of any stray interrupt events***
    - *Unmask interrupt in peripheral*
    - Set bit *within **ienable** control register to enable interrupt*
  - **Other useful API for ISRs**
    - alt_irq_disable_all(void)
    - alt_irq_enable_all(void)
    - alt_ic_irq_enable(alt_u32 ic_id, alt_u32 irq)
    - alt_ic_irq_disable(alt_u32 ic_id, alt_u32 irq)

NIOS II Specifics

- Instruction-Related Exception Procedure
  - **Exceptions occur during execution of Nios II instructions**
    - Trap instruction
      - ***Program issues trap instruction, to generate software trap***
      - ***Issued when program requires servicing by the OS***
    - Break instruction
      - ***Break instruction treated as a break exception***
      - ***Break execution of program, pause***
    - Unimplemented instruction
      - ***Valid instruction that is not implemented in hardware***
    - Illegal instruction
      - ***Instructions with undefined opcode or opcode-extension field***
    - Misaligned data address
    - Misaligned destination address

Multiple Interrupts

- Multiple Interrupts
  - **Which device is requesting service?**
    - Could poll each device to see which one needs servicing
  - **How is appropriate service routine executed?**

  - **Should interrupt nesting be permitted?**
  - **How are two simultaneous requests handled?**
    - Use priority to determine which interrupt to service first

Interrupt Vector

- Interrupt Vector
  - **Reduce service latency because no instructions executed to poll many devices**
  - **Requesting device identifies itself directly with a signal or a binary code (i.e.: vector)**
    - Vectors determine what code is called for each interrupt
    - Allow different code to handled different interrupts
  - **Uses info to find address of ISR in *interrupt-vector table***
    - Table lookup is performed by hardware
    - Vector table located at fixed address, but routines can be located anywhere in memory

Interrupts and Exceptions

- Interrupt Nesting
  - **Usually ISRs run until completion**
  - **Allow interrupts to preempt to reduce latency**

- Prologue
  - **Code inserted before the actual start of function or ISR**
  - **Performs housekeeping such as saving state of the processor before the function/ISR call**
- Epilogue
  - **Code inserted after the end of function or ISR**
  - **Performs housekeeping such as restoring state of the processor before the function/ISR call**